

Assignment 2 · Group

Architectural Evolution of nopCommerce

Software Architectures · Master in Informatics Engineering

Cláudio Teixeira · claudio@ua.pt

Assignment 2 – What You Are Building

Format
Group

Weight
50%

Base System
nopCommerce

Mode
Architecture + Demo

The Core Task

Evolve **nopCommerce** under a demanding business scenario.

You are not rewriting the platform. You are deciding how it should evolve, what should remain inside it, what should move around it, and how the system should behave in a believable operating environment.

What Makes It Architecture

- explicit business drivers
- quality attribute scenarios
- chosen design framework
- documented decisions and rejected alternatives
- visible runtime behavior under degradation and recovery

Deliverable: a runnable architectural demonstration environment, not only diagrams or code fragments.

Three Scenario Paths

SCENARIO A

Federated Commerce After Acquisitions

One group, multiple business units, partial autonomy, and a shared platform challenge.

Think: shared identity, local ERP behavior, catalog variation, local degradation without group collapse.

SCENARIO B

Regulated / Controlled Commerce

Policy, evidence, traceability, approvals, and safe behavior under imperfect verification dependencies.

Think: policy engine, document evidence, approval workflow, degraded verifier, explainable recovery.

SCENARIO C

Omnichannel Commerce Core

noCommerce becomes the commerce core in a wider operational ecosystem.

Think: ERP, warehouse, POS, shipping, stale state, coordination, recovery after inconsistency.

Choose one scenario. The architecture should respond to that scenario, not to every possible problem at once.

What Your Demo Must Prove

Mandatory Shape

- two required use cases for your chosen scenario
- at least two surrounding systems or equivalent external capabilities
- one explicit runtime challenge
- one clear recovery or resolution path

Runtime Challenge Is Not Optional

Your final demonstration must show:

- degraded dependency
- delay, stale state, inconsistency, or failure
- observable system behavior during the problem
- how the architecture contains, routes, or resolves it

SHOW

Normal + stressed operation

PROVE

Recovery, not only failure

AVOID

Happy-path architecture theatre

Architecture Work You Must Show

Design Work

- current-state analysis of nopCommerce
- business drivers and architectural drivers
- bounded contexts and responsibilities
- 3-5 quality attribute scenarios
- target architecture and evolution path

Documentation Work

- choose and apply at least one framework: ADD , ACDM , or ADM
- ADRs with rejected alternatives
- cross-cutting concerns
- evidence pack
- traceability from drivers to implementation

This is a software architecture assignment. Documentation is part of the product, not an appendix.

Scope, Realism, and Creative Freedom

Required Technical Shape

- at least one asynchronous workflow
- at least one explicit reliability decision
- at least two surrounding systems
- at least one independently deployable subsystem, if justified
- no shared database shortcut across extracted boundaries

You May Still Be Creative

- use the named systems or similar
- use real systems, surrogates, stubs, or mock services
- add constraints or stakeholders
- make honest scope cuts

But the original scenario and pressure point must remain clearly visible.

Smaller and defensible beats bigger and vague.

Checkpoint and Final Delivery

Part 1 – Architecture Checkpoint

- scenario choice
- current-state analysis
- domain and boundary model
- QA scenarios
- chosen framework
- target architecture
- at least 3 ADRs
- risk plan and feasibility spike

Part 2 – Final Delivery

- runnable implementation
- architecture report
- updated ADRs
- evidence pack
- live demonstration with degraded or recovering behavior
- defense of tradeoffs and limits

Full brief: [Google Doc](#)

What Strong Work Looks Like

Strong

- one clear architectural problem
- focused evolution
- explicit tradeoffs
- believable runtime challenge and recovery
- tight traceability
- honest defense of limitations

Weak

- too many technologies with shallow purpose
- feature accumulation without architectural tension
- diagrams disconnected from runtime behavior
- vague resilience claims without evidence
- demo that avoids the hard part

This work is **Assignment 2 (group, 50%)** and is meant to show architectural judgment in a realistic operating context.

Dates and Format

Part 1 – Quick Presentation Round

- **Dates:** 05 and 06 May 2026
- **Format:** quick architecture checkpoint presentation
- **Time:** 7 minutes per group

Part 2 – Final Presentation

- **Dates:** 02 and 03 June 2026
- **Format:** full presentation with live demo and architectural defense
- **Time:** 15 minutes per group

Come prepared to explain both what you built and why the architecture evolved that way.